```
export MINIO_ROOT_USER=admin
export MINIO_ROOT_PASSWORD='ChangeThisToAStrongPassword!'

# Region (optional, but recommended for S3 API compatibility)
export MINIO_REGION_NAME=us-east-1

# Enable Prometheus metrics without authentication (internal
networks only)
export MINIO_PROMETHEUS_AUTH_TYPE=public

# KMS / server-side encryption (optional)
export MINIO_KMS_SECRET_KEY='my-minio-key:BASE64_ENCODED_32_
BYTE_KEY'

# Browser console toggle
export MINIO_BROWSER=on
```

For Ceph, most cluster-wide configuration lives in *ceph.conf* and the Ceph config database (ceph config set ...) rather than environment variables, but the following are commonly set at the shell level for CLI convenience:

```
export CEPH_CONF=/etc/ceph/ceph.conf
export CEPH_KEYRING=/etc/ceph/ceph.client.admin.keyring
```

Once the installation of Ceph is complete, you can verify it like this:

```
# Overall cluster health
sudo ceph -s
# Expect: health: HEALTH_OK

# Check OSD status
sudo ceph osd tree

# Check the RGW (S3) endpoint is responding
curl http://<RGW_HOST>:80
```

MinIO can be verified as follows:

```
# Start the server (foreground, for verification)
minio server /mnt/disk{1...4}/minio --console-address ":9001"

# In another terminal, configure an mc alias
mc alias set local http://localhost:9000 admin
'ChangeThisToAStrongPassword!'

# Verify connectivity and list buckets
mc admin info local
```

A healthy mc admin information response reports all drives online and cluster status as 'ok'; a healthy ceph -s reports HEALTH_OK with all PGs (placement groups) active+clean.

Configuration

Let's look at a ceph pool and erasure coding setup. We use an example of an 8PB archive pool using erasure coding for storage efficiency.

```
# Create an erasure-coded profile (6 data chunks, 3 parity
chunks)
sudo ceph osd erasure-code-profile set ec-6-3 k=6 m=3

# Create the pool using that profile
sudo ceph osd pool create archive-pool 128 128 erasure ec-6-3

# Enable the pool for RGW/object storage use
sudo ceph osd pool application enable archive-pool rgw
```

MinIO bucket policy and lifecycle configuration can be done as follows:

```
# Create a bucket
mc mb local/training-datasets

# Set a bucket policy (public read for a specific prefix,
private otherwise)
mc anonymous set download local/training-datasets/public-
samples

# Configure lifecycle rules (e.g., expire temp uploads after 7
days)
mc ilm rule add local/training-datasets \
  --expire-days 7 \
  --prefix "tmp/"

# Enable versioning for reproducibility of training data
snapshots
mc version enable local/training-datasets
```

Example configuration for ML training

A common pattern for large scale ML training is to store sharded datasets (e.g., WebDataset *.tar* shards or Parquet files) in MinIO or Ceph RGW, and stream them directly into training jobs without staging to local disk.

```
Bucket layout is:
training-datasets/
├── imagenet-1k/
│   ├── train/shard-00000.tar ... shard-01023.tar
│   ├── val/shard-00000.tar ... shard-00063.tar
│   └── metadata.json
└── checkpoints/
    └── run-2026-07-01/
```